

Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет __информационных технологий____
Кафедра «_Информационные системы и технологии__»

Направление подготовки/ специальность: _Программное обеспечение игровой
компьютерной индустрии_

ОТЧЕТ

по проектной практике

Студент: __Рябчук Иван Денисович______ Группа: __251-337__

Место прохождения практики: Московский Политех, кафедра
Информационные системы и технологии

Отчет принят с оценкой _____ Дата _____

Руководитель практики: __Дагаев Александр Евгеньевич _____

Москва 2026

Оглавление

Введение:	3
Общая информация о проекте	3
Название проекта:	3
Цель проекта и задачи проекта:	3
Общая характеристика деятельности организации (заказчика проекта)	4
Наименование заказчика:	4
Организационная структура:	5
Описание деятельности:	6
Описание задания по проектной практике:	6
Описание достигнутых результатов по проектной практике:	9
Разработка пользовательского интерфейса:	10
Реализация игровой карты и перемещения:	11
Реализация системы игрового персонажа:	13
Реализация инвентаря и системы создания предметов:	14
Реализация системы квестов:	15
Реализация системы боевых столкновений:	17
Реализация системы торговли:	19
Тестирование разработанного приложения:	20
Итоговый результат разработки:	21
Заключение	22
Список литературы	23
Приложение	24

Введение:

Учебная практика является важной частью подготовки специалистов в области разработки программного обеспечения, поскольку позволяет закрепить теоретические знания и получить практический опыт создания программных приложений.

В рамках учебной практики был разработан проект SimpleRPG2026 — простая ролевая компьютерная игра, реализованная на языке программирования C# с использованием технологии WPF. Проект предназначен для демонстрации основных принципов разработки пользовательских приложений и реализации игровой логики.

В процессе разработки были реализованы пользовательский интерфейс игры, игровая карта, система характеристик персонажа, инвентарь, система квестов, боевые столкновения с противниками, торговля с игровыми торговцами, получение опыта и золота, повышение уровня персонажа, а также механика смерти и возрождения.

Актуальность разработки заключается в возможности применения полученных в ходе обучения знаний языка C# и технологии WPF для создания полноценного интерактивного приложения, содержащего взаимосвязанные программные системы.

Общая информация о проекте

Название проекта: SimpleRPG2026

Цель проекта и задачи проекта:

Разработка простого ролевого игрового приложения на языке C# с использованием технологии WPF, реализующего основные игровые механики, включая перемещение по игровой карте, взаимодействие с противниками и торговцами, управление персонажем, выполнение квестов, использование инвентаря и развитие персонажа.

Для достижения поставленной цели в проекте необходимо было реализовать следующие основные задачи:

1. Разработать пользовательский интерфейс RPG-приложения с использованием WPF.
2. Реализовать игровую карту, состоящую из 17 доступных для перемещения локаций.
3. Реализовать систему характеристик и развития игрового персонажа.
4. Реализовать систему инвентаря и создания предметов по рецептам.
5. Разработать систему квестов с условиями получения и выполнения заданий.
6. Реализовать систему взаимодействия игрока с противниками и проведения боевых столкновений.
7. Реализовать систему торговли с внутриигровыми торговцами.
8. Реализовать механики получения опыта, золота и предметов.
9. Реализовать систему смерти и возрождения персонажа.
10. Обеспечить динамическое отображение элементов управления в зависимости от текущего состояния игры.

В результате выполнения указанных задач должно быть создано интерактивное приложение, объединяющее перечисленные игровые системы в единую программную систему.

Общая характеристика деятельности организации (заказчика проекта)

Наименование заказчика:

Проект SimpleRPG2026 является учебным проектом, выполненным в рамках учебной практики. В связи с этим внешний коммерческий заказчик у проекта отсутствует.

Разработка проекта осуществлялась в соответствии с учебным заданием, направленным на получение практических навыков разработки программного обеспечения с использованием языка C# и технологии WPF.

В качестве образовательной организации, в рамках которой выполнялась разработка, выступает Федеральное государственное автономное образовательное учреждение высшего образования «Московский Политехнический Университет».

Разработка выполнялась студентом направления подготовки «Программное обеспечение игровой компьютерной индустрии» факультета информационных технологий кафедры «Информационные системы и технологии».

Организационная структура:

В связи с учебным характером проекта разработка не осуществлялась в рамках отдельной коммерческой организации или производственного предприятия. Работа выполнялась в образовательной среде Московского Политехнического Университета в рамках учебной практики.

Организационно образовательный процесс включает университет, факультет информационных технологий и кафедру информационных систем и технологий. В рамках учебной практики студент выполняет поставленное учебное задание, осуществляет разработку программного продукта и представляет результаты выполненной работы.

В процессе выполнения проекта студент выступает непосредственно разработчиком программного продукта, выполняя анализ требований, проектирование, программную реализацию и проверку работоспособности созданного приложения.

Описание деятельности:

Московский Политехнический Университет осуществляет образовательную деятельность по различным направлениям подготовки, в том числе связанным с информационными технологиями и разработкой программного обеспечения.

В рамках направления подготовки «Программное обеспечение игровой компьютерной индустрии» осуществляется подготовка специалистов, обладающих знаниями и практическими навыками в области разработки программного обеспечения, компьютерных игр и связанных с ними информационных технологий.

Учебная практика является частью образовательного процесса и направлена на закрепление полученных теоретических знаний посредством выполнения практических заданий. В рамках практики студент получает возможность применить знания языков программирования, технологий разработки пользовательских интерфейсов и принципов построения программных систем при создании собственного программного проекта.

Проект SimpleRPG2026 был выполнен именно в рамках такой практической деятельности. Для разработки приложения использовалась среда Microsoft Visual Studio 2026, язык программирования C# и технология WPF.

Описание задания по проектной практике:

В рамках проектной практики была поставлена задача разработать простое ролевое игровое приложение SimpleRPG2026 на языке программирования C# с использованием технологии WPF. Разрабатываемое приложение должно представлять собой интерактивную игру, в которой пользователь управляет игровым персонажем, исследует доступные локации, взаимодействует с противниками и торговцами, выполняет квесты и получает различные игровые награды.

Одним из основных требований к проекту являлась разработка пользовательского интерфейса, обеспечивающего отображение текущего состояния игрового мира и предоставляющего пользователю необходимые элементы управления. Интерфейс должен содержать информацию об игровом персонаже, его инвентаре, доступных квестах, текущей локации и противнике. Также необходимо предусмотреть область, в которой отображаются последние игровые события и действия пользователя.

Информационный блок персонажа должен отображать его имя, класс, текущее количество очков здоровья, количество золота, накопленный опыт и текущий уровень. В проекте предусмотрена возможность изменения указанных характеристик в зависимости от действий игрока. В частности, здоровье персонажа может уменьшаться в результате получения урона от противников, а опыт и золото увеличиваются после выполнения определённых игровых действий.

Для хранения и использования игровых предметов необходимо было разработать систему инвентаря. В инвентаре должны отображаться имеющиеся у игрока предметы, их количество и стоимость. Игрок получает предметы в качестве награды за победу над противниками и выполнение квестов, может приобретать их у торговцев, а также создавать некоторые предметы с помощью рецептов. Предметы могут расходоваться при использовании или создании других предметов и удаляться из инвентаря при продаже.

Отдельной частью задания являлась разработка системы рецептов. Она должна предоставлять возможность просмотра доступных рецептов и создания игровых предметов при наличии необходимых ресурсов. Созданные предметы добавляются в инвентарь игрока.

Игровой мир должен состоять из 17 доступных для перемещения локаций. Каждая локация может иметь собственное название, описание и

изображение. В зависимости от конкретной локации на ней могут находиться противник, торговец или объект, связанный с выполнением квеста. Для перемещения между локациями должны использоваться кнопки направлений. Доступность кнопок перемещения определяется расположением текущей локации на игровой карте.

В проекте также должна быть реализована система квестов. Игрок может получать задания на определённых локациях. Некоторые квесты доступны только после выполнения определённых условий. После выполнения поставленного задания игрок получает соответствующие награды, которыми могут являться опыт, золото и игровые предметы. Состояние выполнения квеста должно отображаться в пользовательском интерфейсе.

Важной частью игрового процесса является система взаимодействия с противниками. На отдельных локациях игрок может встретить одного из трёх предусмотренных типов врагов. Каждый противник обладает собственными характеристиками здоровья и урона, а также наградами за победу над ним. В качестве награды могут выступать опыт, золото и игровые предметы.

При встрече с противником должна запускаться система боевого столкновения. Игрок и противник поочерёдно наносят друг другу урон до тех пор, пока здоровье одной из сторон не достигнет нулевого значения. Игрок также может покинуть локацию, прекратив столкновение. Во время боя должна сохраняться возможность использовать доступные лечащие предметы из инвентаря.

При победе над противником необходимо начислить игроку предусмотренные награды. Полученный опыт влияет на дальнейшее развитие персонажа, а полученное золото может использоваться для приобретения предметов у торговцев. Предметы, полученные после победы, должны добавляться в инвентарь игрока.

Также требовалось реализовать систему характеристик игрового персонажа. В рамках проекта предусмотрены имя, класс, здоровье, золото, опыт и уровень. На текущем этапе разработки доступен один игровой класс — Fighter. Максимальное количество здоровья персонажа зависит от его уровня. При достижении определённого количества опыта персонаж получает новый уровень, вследствие чего увеличивается его максимальное здоровье.

При снижении текущего здоровья до нуля персонаж считается погибшим. Для продолжения игрового процесса необходимо реализовать механизм возрождения, при котором персонаж возвращается на стартовую локацию, а его здоровье восстанавливается до максимального значения.

Последней основной системой проекта является торговля. Торговля должна быть доступна на локациях, где присутствует игровой торговец. В специальном окне пользователь может просматривать инвентарь персонажа и набор товаров торговца. Игроку предоставляется возможность продавать имеющиеся предметы и получать за них золото, а также приобретать доступные товары торговца для дальнейшего использования в игре.

Таким образом, задание по проектной практике предусматривало разработку взаимосвязанной системы игровых механик, объединённых единым пользовательским интерфейсом. Основное внимание при разработке уделялось реализации игровой логики, взаимодействию между отдельными системами приложения и корректному отображению текущего состояния игры.

Описание достигнутых результатов по проектной практике:

В результате выполнения проектной практики было разработано игровое приложение SimpleRPG2026, реализующее основные механики простой ролевой игры. Приложение было создано на языке

программирования C# с использованием технологии WPF и среды разработки Microsoft Visual Studio 2026.

В процессе разработки были реализованы пользовательский интерфейс игры, игровая карта, система характеристик персонажа, инвентарь, рецепты создания предметов, квестовая система, боевые столкновения с противниками и система торговли.

Разработка пользовательского интерфейса:

Одним из первых этапов разработки было создание основного пользовательского интерфейса приложения. Интерфейс был разделён на несколько функциональных областей, каждая из которых отвечает за определённую часть игрового процесса.

В левой части окна расположена область с информацией о текущем персонаже. В ней отображаются его имя, класс, количество очков здоровья, золото, накопленный опыт и текущий уровень. Благодаря этому игрок может постоянно отслеживать основные характеристики своего персонажа.

Ниже располагается область инвентаря. Она содержит список имеющихся у игрока предметов, их количество и стоимость. Инвентарь также содержит вкладку с рецептами, предназначенную для создания новых игровых предметов.

В нижней левой части окна находится область квестов. В ней отображается название текущего задания и информация о его выполнении.

Центральная часть интерфейса предназначена для отображения игровых событий. В ней выводятся сообщения о перемещении, получении и использовании предметов, нанесении урона, победе над противником, получении опыта и золота, а также другие события, происходящие во время игры.

В правой части окна отображается информация о текущей игровой локации. Пользователю показываются название и описание локации, а также соответствующее изображение. При наличии противника дополнительно отображается информация о нём и его текущем количестве очков здоровья.

В нижней части окна размещены элементы управления персонажем. Доступные направления перемещения отображаются в зависимости от текущего положения персонажа на карте. Также в зависимости от состояния игры могут отображаться кнопки использования предметов и взаимодействия с другими объектами.

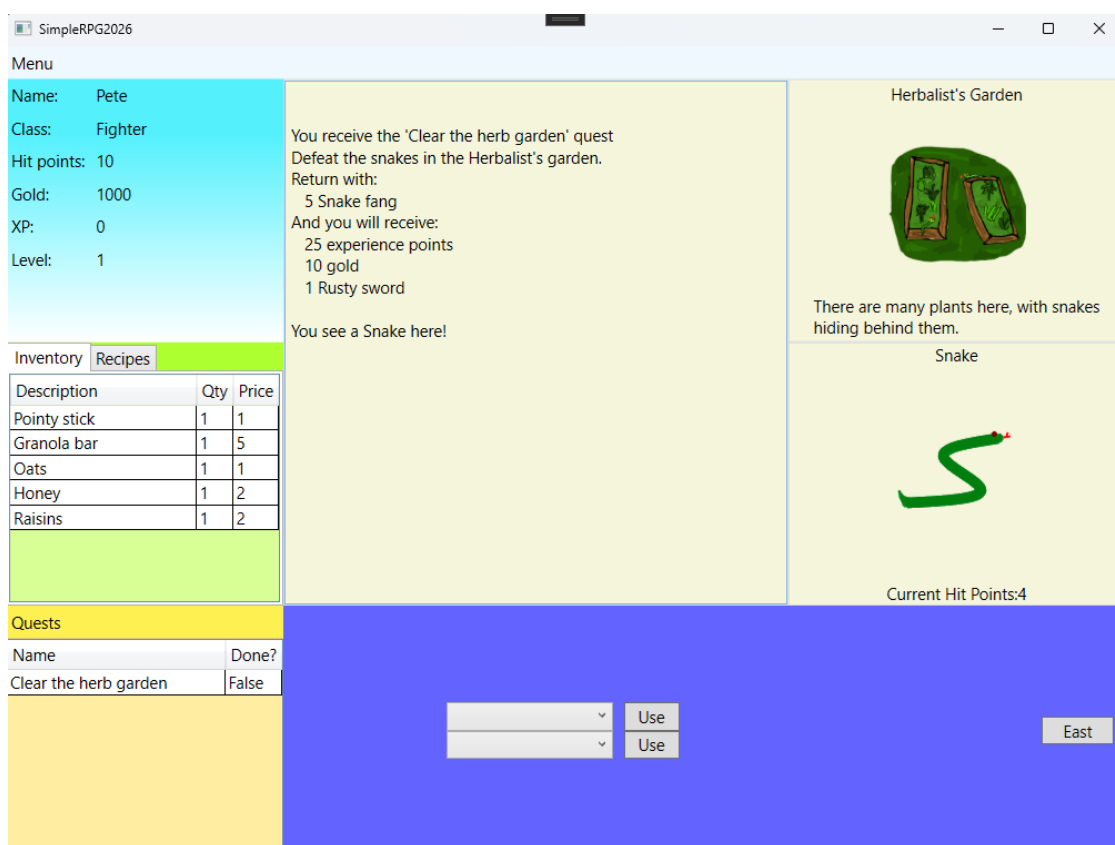


Рисунок 1 – Основное игровое окно приложения SimpleRPG2026

Реализация игровой карты и перемещения:

В рамках проекта была реализована игровая карта, состоящая из 17 доступных локаций. Локации связаны между собой и образуют игровой маршрут, по которому может перемещаться персонаж.

Для управления перемещением используются кнопки направлений: север, юг, запад и восток. Набор отображаемых кнопок изменяется в зависимости от текущего положения игрока. Таким образом, пользователю доступны только те направления, в которых действительно существует переход на другую локацию.

Каждая игровая локация обладает собственным названием, описанием и изображением. Кроме того, различные локации могут содержать противников, торговцев или элементы, связанные с выполнением квестов.

В процессе тестирования были проверены переходы между различными локациями и изменение отображаемой информации после перемещения персонажа.

```
<?xml version="1.0" encoding="utf-8" />
<Locations RootImagePath="/Images/Locations/">
  <Location X="-3" Y="-1" Name="Bandit Camp" ImageName="BanditCamp.png">
    <Description><![CDATA[]]></Description>
  </Location>
  <Location X="-2" Y="-1" Name="Meadows" ImageName="Meadows.png">
    <Description><![CDATA[Tall grass everywhere, with giant rats hiding between them.]]></Description>
    <Monsters>
      <Monster ID="1" Percent="20"/>
      <Monster ID="2" Percent="80"/>
    </Monsters>
  </Location>
  <Location X="-1" Y="-1" Name="Guard Camp" ImageName="Guard'sOutpost.png">
    <Description><![CDATA[Guards training here.]]></Description>
    <Trader ID="5"/>
    <Quests>
      <Quest ID="2"/>
    </Quests>
  </Location>
  <Location X="0" Y="-1" Name="Graveyard" ImageName="Graveyard.png">
    <Description><![CDATA[Spooky place...]]></Description>
  </Location>
  <Location X="-1" Y="0" Name="Some local houses" ImageName="InhabitedHouses.png">
    <Description><![CDATA[Just local houses.]]></Description>
  </Location>
  <Location X="-1" Y="1" Name="Town Square" ImageName="TownSquare.png">
    <Description><![CDATA[]]></Description>
  </Location>
  <Location X="0" Y="0" Name="Church" ImageName="Church.png">
    <Description><![CDATA[The place you first appeared at with a quest in mind to bring peace in this world.]]></Description>
  </Location>
  <Location X="1" Y="-1" Name="Dense forest" ImageName="BarelyVisiblePath.png">
    <Description><![CDATA[Dense forest with a barely visible path in sight]]></Description>
    <Monsters>
      <Monster ID="3" Percent="100"/>
    </Monsters>
  </Location>
  <Location X="0" Y="1" Name="Market" ImageName="Marketplace.png">
    <Description><![CDATA[Local market.]]></Description>
    <Trader ID="1"/>
  </Location>
  <Location X="0" Y="2" Name="Weapon blacksmith's house" ImageName="WeaponBlacksmith'sHouse.png">
    <Description><![CDATA[Local blacksmith.]]></Description>
    <Trader ID="4"/>
  </Location>
</Locations>
```

Рисунок 2 – Отображение игровой локации и доступных направлений перемещения

Реализация системы игрового персонажа:

В приложении была реализована система характеристик игрового персонажа. В текущей версии игры персонаж имеет имя Pete и класс Fighter.

Персонаж обладает следующими основными характеристиками:

- очки здоровья;
- максимальное количество здоровья;
- золото;
- опыт;
- уровень.

Количество текущего здоровья изменяется в зависимости от полученного персонажем урона. При использовании соответствующих предметов здоровье может восстанавливаться, при этом оно не может превышать установленное максимальное значение.

За победу над противниками и выполнение квестов персонаж получает опыт. После достижения необходимого количества опыта происходит повышение уровня. Увеличение уровня влияет на максимальное количество здоровья персонажа.

Также была реализована система получения золота. Золото может быть получено в результате победы над противниками, выполнения квестов и продажи предметов торговцу. Полученные средства используются для приобретения новых предметов.

При снижении количества здоровья до нуля персонаж считается погибшим. После смерти предусмотрено его возрождение с возвращением на стартовую локацию и восстановлением здоровья до максимального значения.

Реализация инвентаря и системы создания предметов:

В приложении реализован инвентарь, предназначенный для хранения игровых предметов. Для каждого предмета отображаются его название, количество и стоимость.

Игрок может получать предметы различными способами: в качестве награды за выполнение квестов, после победы над противниками, при покупке у торговца или в результате создания с использованием рецептов.

В интерфейсе инвентаря предусмотрена отдельная вкладка Recipes, содержащая доступные рецепты. При наличии необходимых компонентов игрок может создать соответствующий предмет. Использованные для создания ресурсы удаляются из инвентаря, а полученный предмет добавляется в него.

Кроме того, некоторые предметы могут использоваться непосредственно во время игрового процесса. В частности, лечащие предметы позволяют восстанавливать здоровье персонажа, в том числе во время боевого столкновения.

Inventory

Recipes

Description	Qty	Price
Pointy stick	1	1
Granola bar	1	5
Oats	1	1
Honey	1	2
Raisins	1	2

```
<?xml version="1.0" encoding="utf-8" ?>
<Recipes>
  <Recipe ID="1">
    <Name><![CDATA[Granola bar]]></Name>
    <Ingredients>
      <Item ID="3001" Quantity="1"/>
      <Item ID="3002" Quantity="1"/>
      <Item ID="3003" Quantity="1"/>
    </Ingredients>
    <OutputItems>
      <Item ID="2001" Quantity="1"/>
    </OutputItems>
  </Recipe>
  <Recipe ID="2">
    <Name><![CDATA[Good Sword]]></Name>
    <Ingredients>
      <Item ID="1002" Quantity="1"/>
      <Item ID="3004" Quantity="1"/>
    </Ingredients>
    <OutputItems>
      <Item ID="1003" Quantity="1"/>
    </OutputItems>
  </Recipe>
</Recipes>
```

Рисунок 3 и Рисунок 4 – Инвентарь и рецепты создания предметов

Реализация системы квестов:

В проекте была разработана система квестов, позволяющая получать и выполнять игровые задания.

Квесты выдаются на определённых локациях. Для некоторых заданий предусмотрены дополнительные условия получения или выполнения. После выполнения необходимых действий состояние соответствующего квеста изменяется.

В качестве примера реализован квест «Clear the herb garden», целью которого является победа над змеями в саду травника. После выполнения необходимых условий игрок получает награды в виде опыта, золота и игрового предмета.

Также в игре реализован квест «Clear clean path in the forest», связанный с победой над противниками в лесной локации. После его выполнения в интерфейсе отображается соответствующий статус, а персонажу начисляются предусмотренные награды.

Состояние квестов отображается в отдельной таблице, где пользователь может увидеть название задания и информацию о том, выполнено оно или

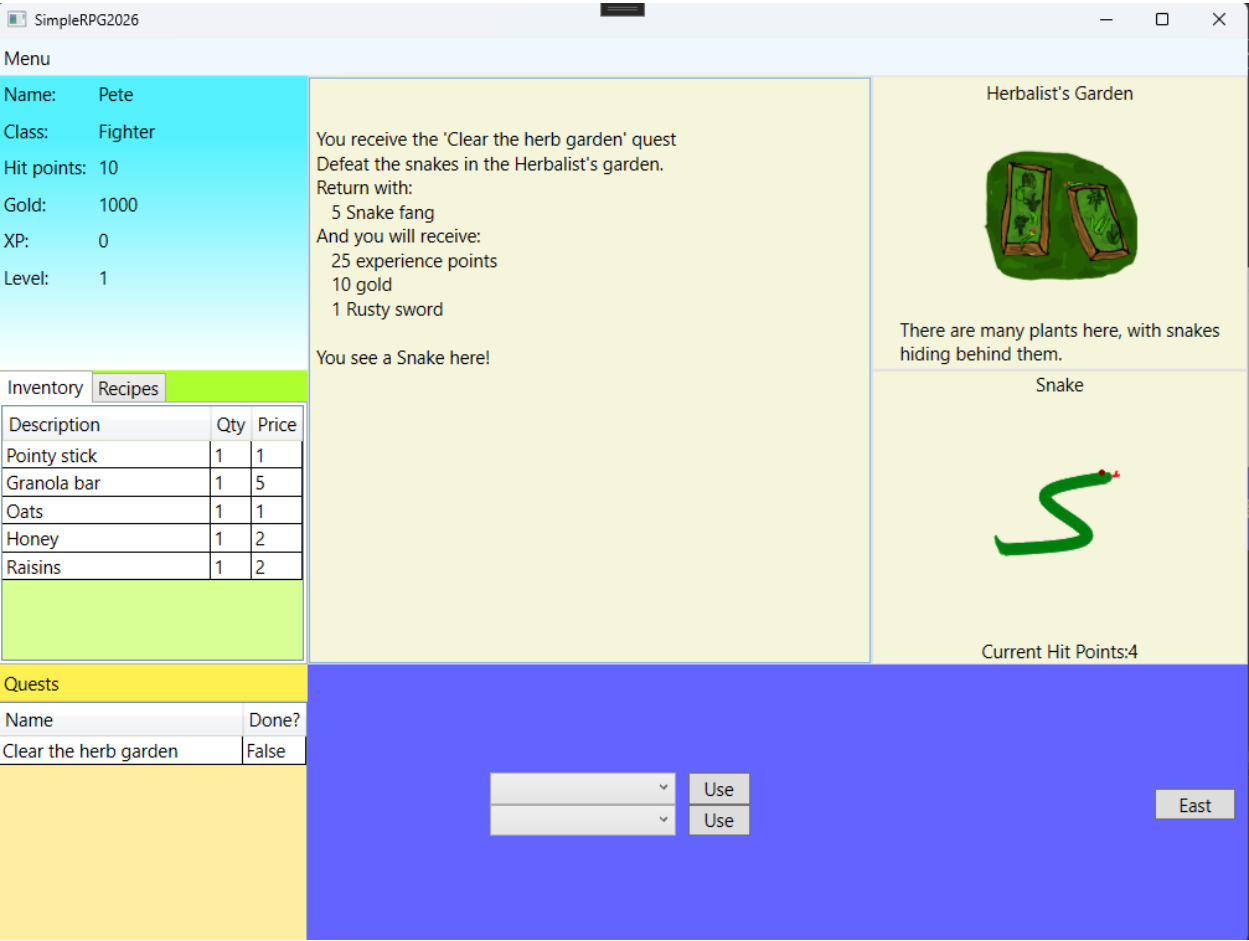


Рисунок 5 – Отображение активного квеста

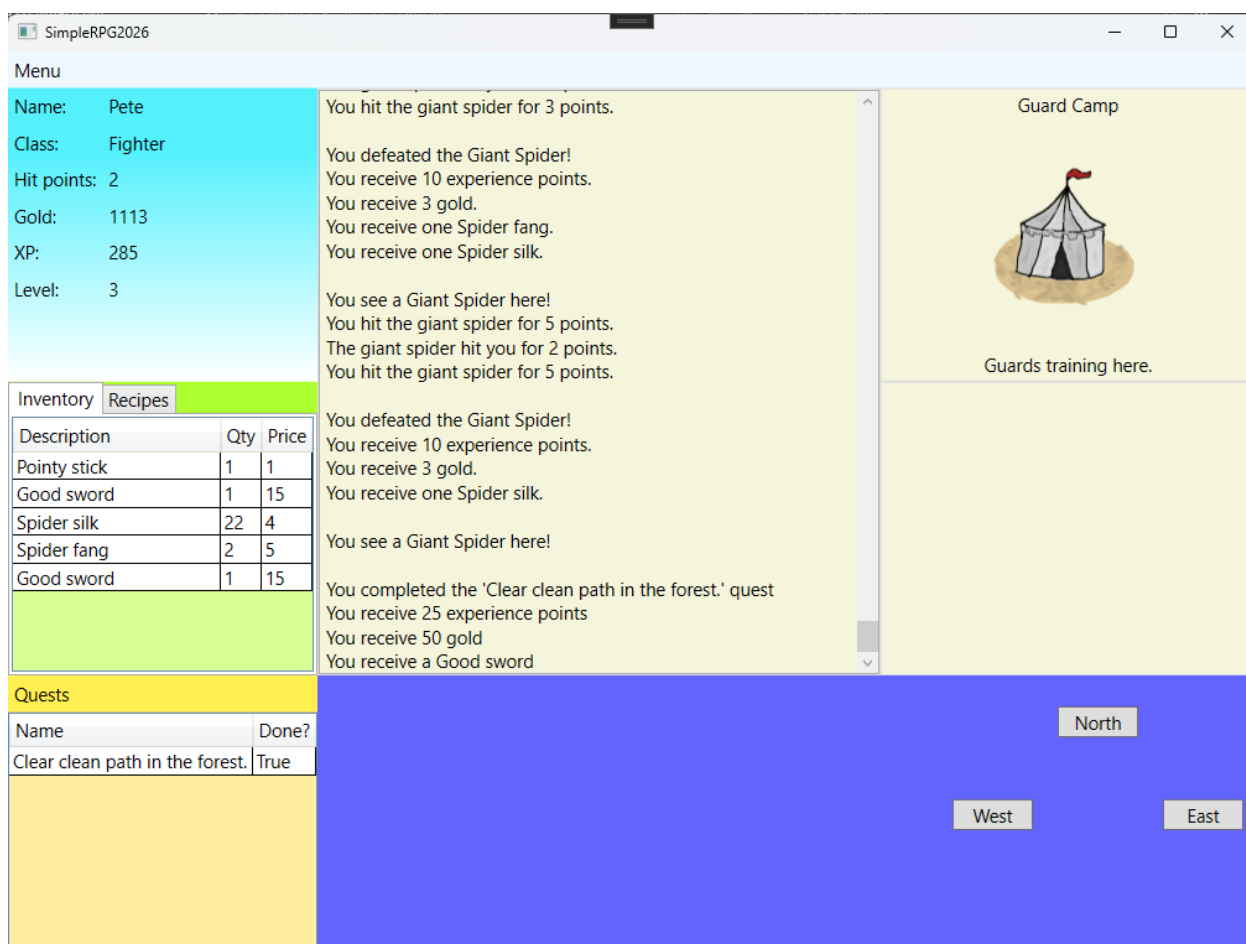


Рисунок 6 – Отображение выполненного квеста

Реализация системы боевых столкновений:

Одной из основных реализованных игровых механик стала система боевых столкновений.

На определённых локациях игрок может встретить противника. В процессе разработки были предусмотрены различные типы врагов, обладающие собственными характеристиками здоровья и урона, а также индивидуальными наградами.

После встречи с противником игрок получает возможность атаковать его с помощью доступного оружия. Бой проходит поочерёдно: игрок наносит противнику урон, после чего противник выполняет ответную атаку. Данное взаимодействие продолжается до тех пор, пока здоровье одной из сторон не достигнет нуля.

Результаты каждого действия отображаются в центральной области игровых событий. Например, пользователь может увидеть сообщения о нанесённом противнику уроне, полученном от него уроне, промахах и результате боя.

После победы над противником игрок получает предусмотренные награды. В зависимости от типа противника награда может включать опыт, золото и предметы. Полученные предметы автоматически добавляются в инвентарь.

В процессе тестирования была проверена возможность использования лечащих предметов во время боя. Это позволяет игроку восстанавливать здоровье персонажа и продолжать сражение.

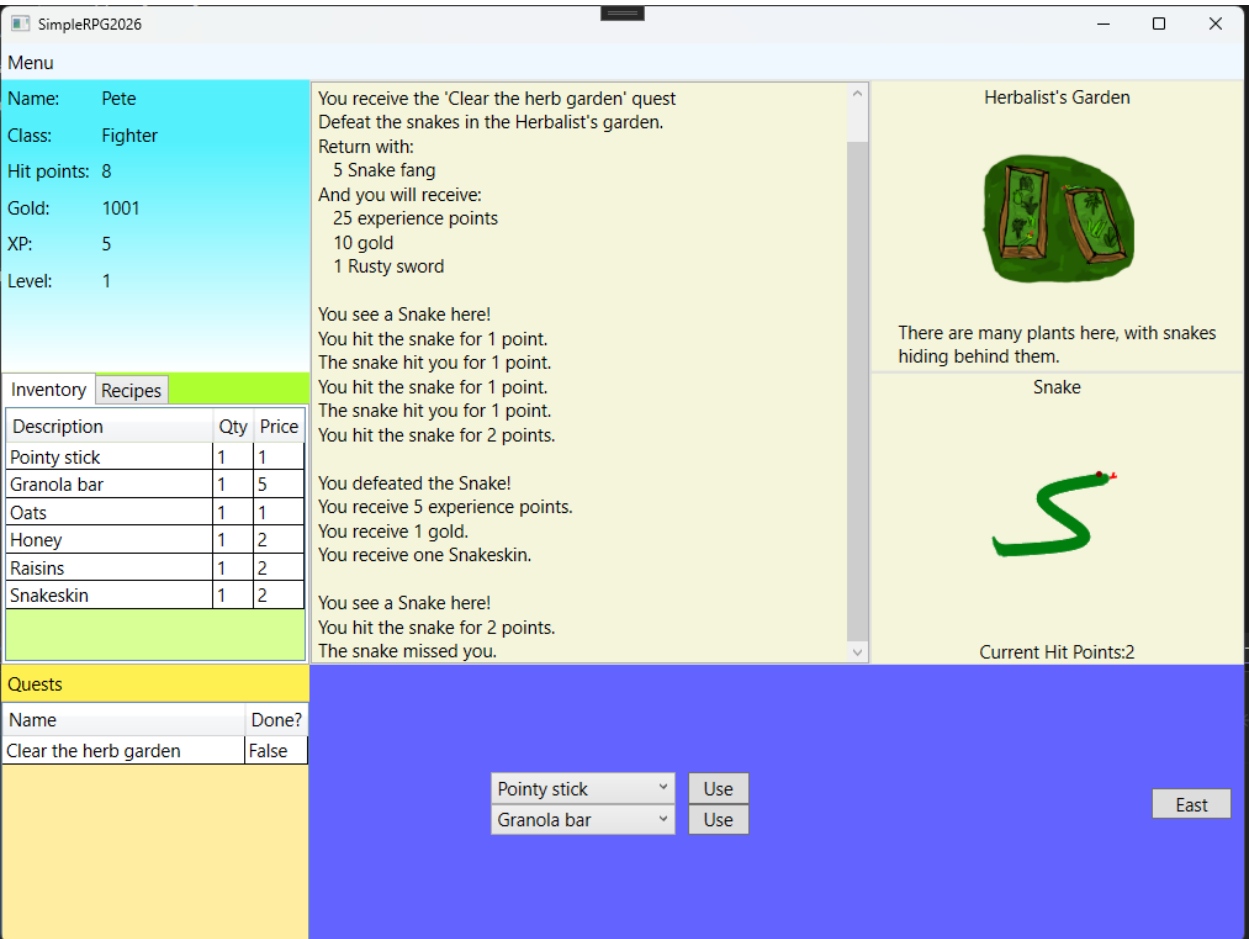


Рисунок 6 – Боестолкновение с противником

На представленном примере видно, что в правой части окна отображается противник и его текущее здоровье, а в центральной области фиксируются результаты атак игрока и противника.

Реализация системы торговли:

В проекте была реализована система торговли с игровыми торговцами. Возможность торговли предоставляется на тех локациях, где присутствует соответствующий игровой персонаж.

При открытии торгового окна пользователь получает доступ к двум спискам: инвентарю игрока и инвентарю торговца. Для каждого предмета отображаются его название, количество и стоимость.

Игрок может продать имеющийся предмет торговцу. В результате продажи количество золота персонажа увеличивается, а проданный предмет удаляется или уменьшается в количестве в его инвентаре.

Также реализована возможность приобретения предметов у торговца. При покупке стоимость товара вычитается из золота игрока, а приобретённый предмет добавляется в его инвентарь.

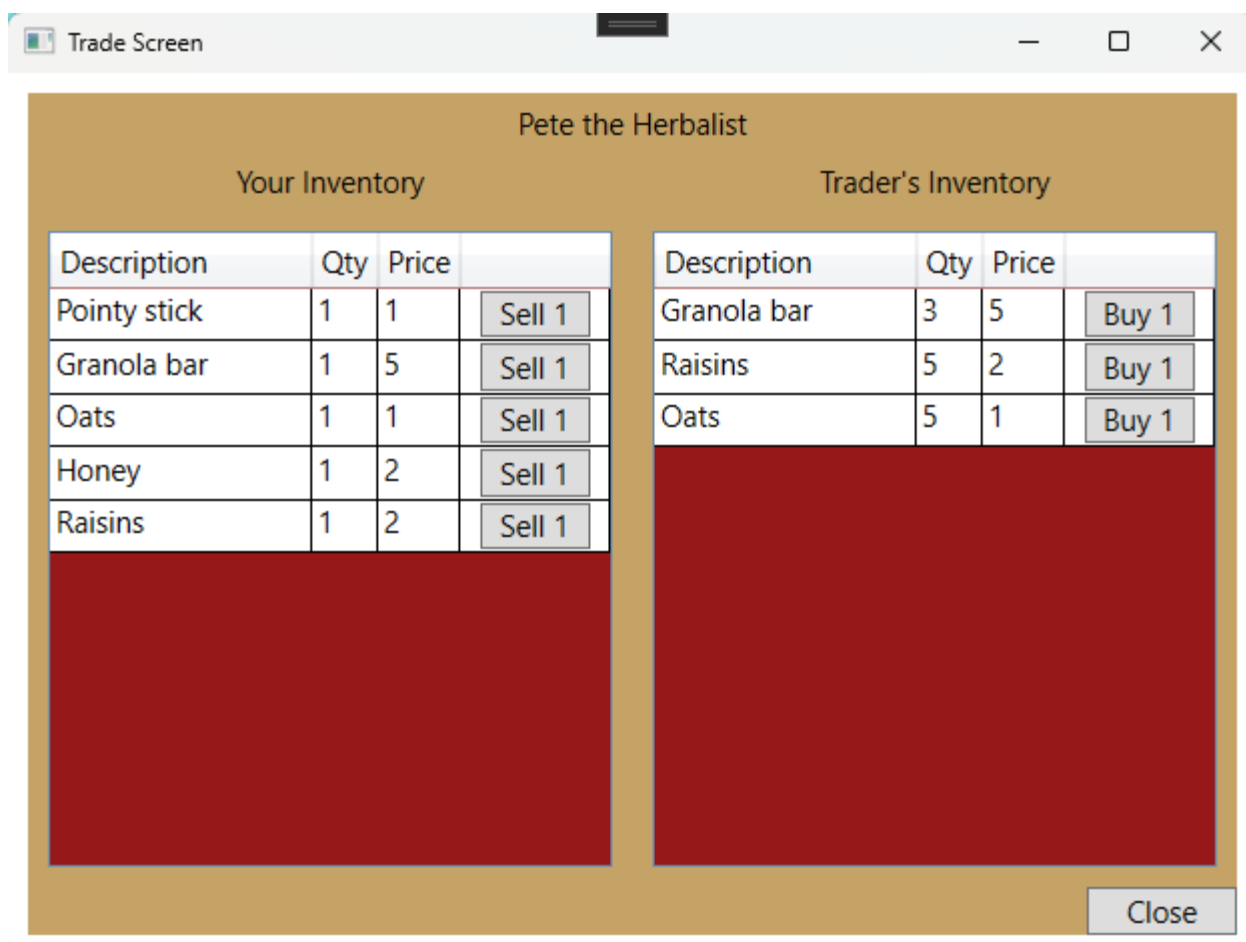


Рисунок 7 – Окно торговли

На рисунке представлено торговое окно, в котором отображаются инвентарь персонажа и товары торговца. Для каждой позиции доступны соответствующие операции покупки или продажи.

Тестирование разработанного приложения:

После завершения реализации основных функций была проведена проверка работоспособности приложения.

В процессе тестирования проверялась корректность основных игровых сценариев: перемещение между локациями, отображение характеристик персонажа, взаимодействие с противниками, нанесение и получение урона, получение наград, выполнение квестов, использование предметов, изменение уровня персонажа, а также покупка и продажа предметов.

Отдельно проверялась корректность изменения состояния игрового интерфейса. При изменении игровой ситуации соответствующие элементы

управления появляются или становятся недоступными в зависимости от текущего состояния персонажа и локации.

В результате тестирования основные реализованные функции приложения продемонстрировали соответствие поставленным требованиям. Игрок может последовательно перемещаться по игровому миру, взаимодействовать с его объектами, развивать персонажа и получать награды за выполнение различных игровых действий.

Итоговый результат разработки:

В результате выполнения проектной практики было создано работоспособное приложение SimpleRPG2026, объединяющее несколько взаимосвязанных игровых систем.

В разработанном приложении реализованы:

- пользовательский интерфейс на основе WPF;
- игровая карта из 17 локаций;
- перемещение между доступными локациями;
- система характеристик персонажа;
- система опыта и уровней;
- система здоровья;
- инвентарь;
- создание предметов по рецептам;
- система квестов;
- система боевых столкновений;
- противники с различными характеристиками;
- получение опыта, золота и предметов за игровые действия;
- использование предметов;
- система смерти и возрождения;
- взаимодействие с торговцами;
- покупка и продажа игровых предметов;
- журналирование последних игровых событий.

Таким образом, разработанный программный продукт реализует основные требования, поставленные в рамках проектной практики, и представляет собой законченное учебное приложение, демонстрирующее

применение языка C# и технологии WPF для создания интерактивных программных систем.

Исходный код разработанного приложения, а также материалы проекта размещены в репозитории GitHub. Репозиторий содержит исходные файлы проекта и позволяет ознакомиться с результатами выполненной разработки.

Репозиторий проекта: <https://github.com/SilurusPete/SimpleRPG2026>

Заключение

В ходе выполнения учебной практики был разработан программный проект SimpleRPG2026 — простая ролевая игра, созданная на языке программирования C# с использованием технологии WPF.

В процессе работы были закреплены и применены на практике знания, полученные при изучении программирования и разработки программных приложений. В рамках проекта был разработан пользовательский интерфейс игры, реализована игровая карта, система перемещения между локациями, характеристики игрового персонажа, инвентарь, рецепты создания предметов, система квестов, боевые столкновения с противниками и система торговли.

В результате разработки было создано приложение, позволяющее пользователю управлять игровым персонажем, исследовать игровой мир, взаимодействовать с противниками и торговцами, выполнять квесты, получать опыт, золото и предметы, повышать уровень персонажа и использовать предметы из инвентаря.

Выполненная работа позволила получить практический опыт разработки приложений с графическим интерфейсом на WPF, работы с языком C#, организации игровой логики и взаимодействия между отдельными компонентами программной системы.

При разработке также использовались учебные материалы сайта SOSCSRPG – Scott's Open-Source C# Role-Playing Game, посвящённые созданию RPG на C# и WPF. Материалы сайта использовались в качестве учебной основы и примера реализации отдельных игровых механик, после чего полученные знания применялись при создании собственного проекта.

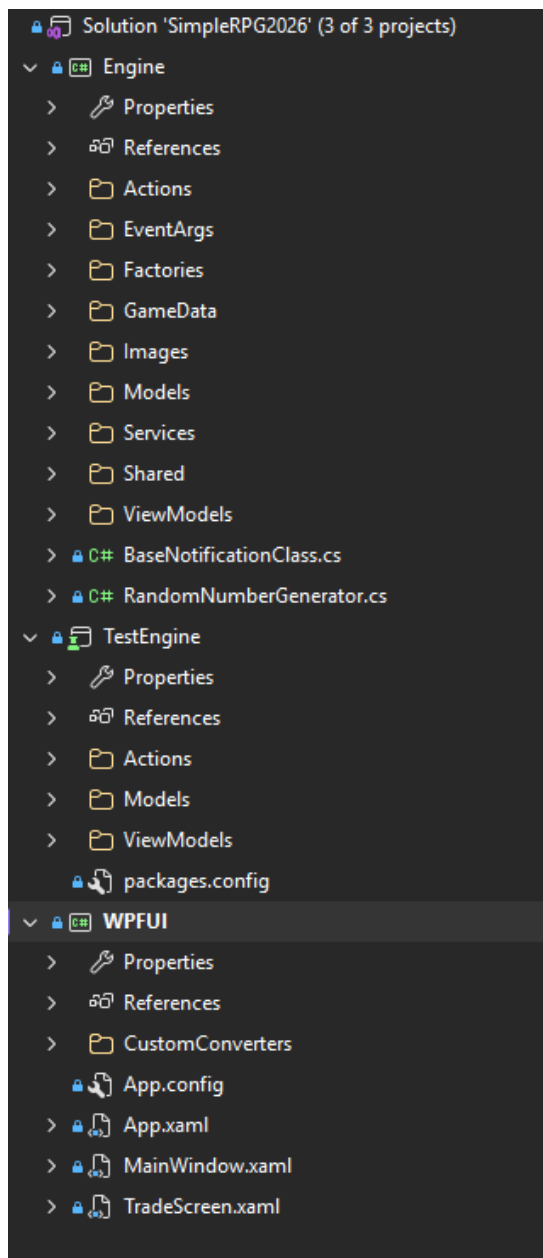
Таким образом, поставленная цель учебной практики была достигнута. Разработанный проект соответствует основным требованиям задания и

демонстрирует возможность применения языка C# и технологии WPF для создания интерактивного игрового приложения.

Список литературы

1. Microsoft. **C# documentation** [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/dotnet/csharp/> (дата обращения: 06.03.2026).
2. Microsoft. **WPF documentation** [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/dotnet/desktop/wpf/> (дата обращения: 06.03.2026).
3. Scott Lilly. **SOSCSRPG – Scott's Open-Source C# Role-Playing Game** [Электронный ресурс]. — Режим доступа: [SOSCSRPG](https://www.soscsrpg.com/) (дата обращения: 16.04.2026).
4. Scott Lilly. **Build a C#/WPF RPG: Lessons** [Электронный ресурс]. — Режим доступа: [Учебные материалы SOSCSRPG](https://www.soscsrpg.com/build-a-c-wpf-rpg-lessons/) (дата обращения: 16.04.2026).
5. SimpleRPG2026. **Исходный код проекта** [Электронный ресурс]. — GitHub. — Режим доступа: <https://github.com/SilurusPete/SimpleRPG2026> (дата обращения: 27.05.2026).

Приложение




```

1  using Engine.Factories;
2  using System.Collections.Generic;
3
4  namespace Engine.Models
5  {
6      12 references
7      public class Monster : LivingEntity
8      {
9          private readonly List<ItemPercentage> _lootTable =
10             new List<ItemPercentage>();
11
12         3 references
13         public int ID { get; }
14         2 references
15         public string ImageName { get; }
16         4 references
17         public int RewardExperiencePoints { get; }
18
19         2 references
20         public Monster(int id, string name, string imageName,
21             int maximumHitPoints,
22             GameItem currentWeapon,
23             int rewardExperiencePoints, int gold) :
24             base(name, maximumHitPoints, maximumHitPoints, gold)
25         {
26             ID = id;
27             ImageName = imageName;
28             CurrentWeapon = currentWeapon;
29             RewardExperiencePoints = rewardExperiencePoints;
30         }
31
32         2 references
33         public void AddItemToLootTable(int id, int percentage)
34         {
35             _lootTable.RemoveAll(ip => ip.ID == id);
36
37             _lootTable.Add(new ItemPercentage(id, percentage));
38         }
39
40         1 reference
41         public Monster GetNewInstance()
42         {
43             Monster newMonster =
44                 new Monster(ID, Name, ImageName, MaximumHitPoints, CurrentWeapon,
45                     RewardExperiencePoints, Gold);
46
47             foreach (ItemPercentage itemPercentage in _lootTable)
48             {
49                 newMonster.AddItemToLootTable(itemPercentage.ID, itemPercentage.Percentage);
50
51                 if (RandomNumberGenerator.NumberBetween(1, 100) <= itemPercentage.Percentage)
52                 {
53                     newMonster.AddItemToInventory(ItemFactory.CreateGameItem(itemPercentage.ID));
54                 }
55             }
56
57             return newMonster;
58         }
59     }
60 }

```

```

1  using Engine.Actions;
2  using Engine.Models;
3  using Engine.Shared;
4  using System.Collections.Generic;
5  using System.IO;
6  using System.Linq;
7  using System.Xml;
8
9  namespace Engine.Factories
10 {
11      28 references
12      public static class ItemFactory
13      {
14          private const string GAME_DATA_FILENAME = ".\\GameData\\GameItems.xml";
15
16          private static readonly List<GameItem> _standardGameItems = new List<GameItem>();
17
18          0 references
19          static ItemFactory()
20          {
21              if (File.Exists(GAME_DATA_FILENAME))
22              {
23                  XmlDocument data = new XmlDocument();
24                  data.LoadXml(File.ReadAllText(GAME_DATA_FILENAME));
25
26                  LoadItemsFromNodes(data.SelectNodes("/GameItems/Weapons/Weapon"));
27                  LoadItemsFromNodes(data.SelectNodes("/GameItems/HealingItems/HealingItem"));
28                  LoadItemsFromNodes(data.SelectNodes("/GameItems/MiscellaneousItems/MiscellaneousItem"));
29              }
30              else
31              {
32                  throw new FileNotFoundException($"Missing data file: {GAME_DATA_FILENAME}");
33              }
34          }
35
36          25 references
37          public static GameItem CreateGameItem(int itemTypeID)
38          {
39              return _standardGameItems.FirstOrDefault(item => item.ItemTypeID == itemTypeID)?.Clone();
40          }
41
42          2 references
43          public static string ItemName(int itemTypeID)
44          {
45              return _standardGameItems.FirstOrDefault(i => i.ItemTypeID == itemTypeID)?.Name ?? "";
46          }
47
48          3 references
49          private static void LoadItemsFromNodes(XmlNodeList nodes)
50          {
51              if (nodes == null)
52              {
53                  return;
54              }
55
56              foreach (XmlNode node in nodes)
57              {
58                  GameItem.ItemCategory itemCategory = DetermineItemCategory(node.Name);
59
60                  GameItem gameItem =
61                      new GameItem(itemCategory,
62                                  node.AttributeAsInt("ID"),
63                                  node.AttributeAsString("Name"),
64                                  node.AttributeAsInt("Price"),
65                                  itemCategory == GameItem.ItemCategory.Weapon);
66
67                  if (itemCategory == GameItem.ItemCategory.Weapon)
68                  {
69                      gameItem.Action =

```

```

63         new AttackWithWeapon(gameItem,
64                               node.AttributeAsInt("MinimumDamage"),
65                               node.AttributeAsInt("MaximumDamage"));
66     }
67     else if (itemCategory == GameItem.ItemCategory.Consumable)
68     {
69         gameItem.Action =
70             new Heal(gameItem,
71                     node.AttributeAsInt("HitPointsToHeal"));
72     }
73     _standardGameItems.Add(gameItem);
74 }
75 }
76
77 1 reference
78 private static GameItem.ItemCategory DetermineItemCategory(string itemType)
79 {
80     switch (itemType)
81     {
82     case "Weapon":
83         return GameItem.ItemCategory.Weapon;
84     case "HealingItem":
85         return GameItem.ItemCategory.Consumable;
86     default:
87         return GameItem.ItemCategory.Miscellaneous;
88     }
89 }
90 }

```

```

<?xml version="1.0" encoding="utf-8" ?>
<GameItems>
  <Weapons>
    <Weapon ID="1001" Name="Pointy stick" Price="1" MinimumDamage="1" MaximumDamage="2"/>
    <Weapon ID="1002" Name="Rusty sword" Price="5" MinimumDamage="1" MaximumDamage="3"/>
    <Weapon ID="1003" Name="Good sword" Price="15" MinimumDamage="2" MaximumDamage="5"/>
    <Weapon ID="1501" Name="Snake fang" Price="0" MinimumDamage="0" MaximumDamage="2"/>
    <Weapon ID="1502" Name="Rat claw" Price="0" MinimumDamage="0" MaximumDamage="2"/>
    <Weapon ID="1503" Name="Spider fang" Price="0" MinimumDamage="0" MaximumDamage="4"/>
  </Weapons>
  <HealingItems>
    <HealingItem ID="2001" Name="Granola bar" Price="5" HitPointsToHeal="2"/>
  </HealingItems>
  <MiscellaneousItems>
    <MiscellaneousItem ID="3001" Name="Oats" Price="1"/>
    <MiscellaneousItem ID="3002" Name="Honey" Price="2"/>
    <MiscellaneousItem ID="3003" Name="Raisins" Price="2"/>
    <MiscellaneousItem ID="9001" Name="Snake fang" Price="1"/>
    <MiscellaneousItem ID="9002" Name="Snakeskin" Price="2"/>
    <MiscellaneousItem ID="9003" Name="Rat tail" Price="1"/>
    <MiscellaneousItem ID="9004" Name="Rat fur" Price="2"/>
    <MiscellaneousItem ID="9005" Name="Spider fang" Price="1"/>
    <MiscellaneousItem ID="9006" Name="Spider silk" Price="2"/>
  </MiscellaneousItems>
</GameItems>

```

```

1  using System;
2  using System.Collections.ObjectModel;
3  using System.Linq;
4
5  namespace Engine.Models
6  {
7      8 references
8      public class Player : LivingEntity
9      {
10         #region Properties
11
12         private string _CharacterClass;
13         private int _Exp;
14
15         1 reference
16         public string CharacterClass
17         {
18             get { return _CharacterClass; }
19             set
20             {
21                 _CharacterClass = value;
22                 OnPropertyChanged();
23             }
24         }
25
26         3 references
27         public int Exp
28         {
29             get { return _Exp; }
30             private set
31             {
32                 _Exp = value;
33                 OnPropertyChanged();
34                 SetLevelAndMaximumHitPoints();
35             }
36         }
37
38         4 references
39         public ObservableCollection<QuestStatus> Quests { get; }
40         3 references
41         public ObservableCollection<Recipe> Recipes { get; }
42
43         #endregion
44
45         public event EventHandler OnLeveledUp;
46
47         1 reference
48         public Player(string name, string characterClass, int experiencePoints,
49                     int maximumHitPoints, int currentHitPoints, int gold) :
50             base(name, maximumHitPoints, currentHitPoints, gold)
51         {
52             CharacterClass = characterClass;
53             Exp = experiencePoints;
54
55             Quests = new ObservableCollection<QuestStatus>();
56             Recipes = new ObservableCollection<Recipe>();
57         }
58
59         2 references
60         public void AddExperience(int experiencePoints)
61         {
62             Exp += experiencePoints;
63         }
64
65         1 reference
66         public void LearnRecipe(Recipe recipe)

```

```

61     {
62         if (!Recipes.Any(r => r.ID == recipe.ID))
63         {
64             Recipes.Add(recipe);
65         }
66     }
67
68     1 reference
69     private void SetLevelAndMaximumHitPoints()
70     {
71         int originalLevel = Level;
72
73         Level = (Exp / 100) + 1;
74
75         if (Level != originalLevel)
76         {
77             MaximumHitPoints = Level * 10;
78
79             OnLeveledUp?.Invoke(this, System.EventArgs.Empty);
80         }
81     }
82 }
83

```

```

1  using System;
2  using System.Xml;
3
4  namespace Engine.Shared
5  {
6      0 references
7      public static class ExtensionMethods
8      {
9          33 references
10         public static int AttributeAsInt(this XmlNode node, string attributeName)
11         {
12             return Convert.ToInt32(node.AttributeAsString(attributeName));
13         }
14
15         8 references
16         public static string AttributeAsString(this XmlNode node, string attributeName)
17         {
18             XmlAttribute attribute = node.Attributes?[attributeName];
19
20             if (attribute == null)
21             {
22                 throw new ArgumentException($"The attribute '{attributeName}' does not exist");
23             }
24
25             return attribute.Value;
26         }
27     }
28 }

```